

# SmartJourney: Project Summary, Workflows and Demo Guide

Prepared 2026-10-02 for the progress evaluation. It draws on `REPO_STATUS.md` (surveyed 2026-09-23), `RUNNING.md`, the three CI workflow files, the Master Test Plan and the project notes. Items marked **verify** are things I could not confirm from the files, so check them before you say them out loud.

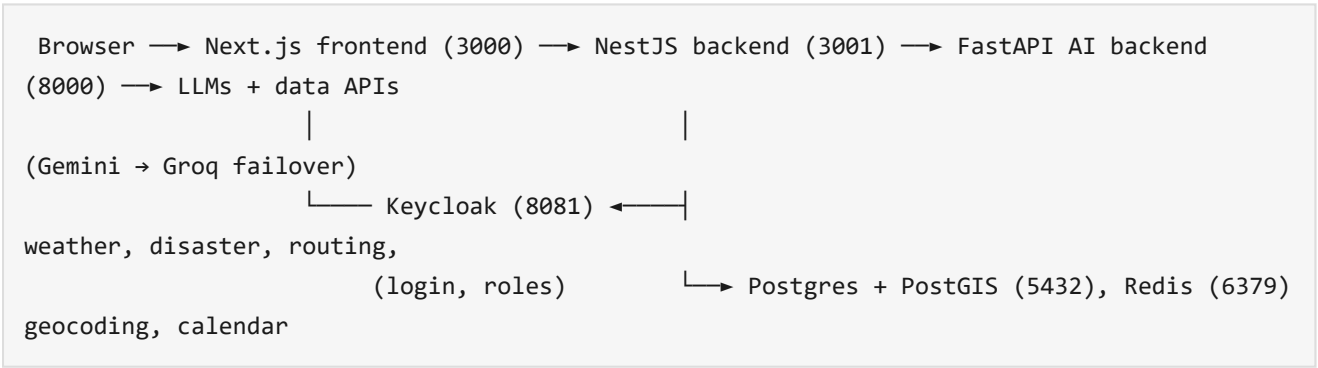
## 1. What SmartJourney is

An AI trip-planning system for **Sri Lanka**. A traveler chats with the system ("3 days, Kandy and Ella, 50,000 LKR, love hiking"). A multi-agent AI backend produces a day-by-day itinerary using real data (listings, weather, disaster alerts, routing, budget) and stays within budget. The traveler can then: - refine the plan by chat ("make day 2 cheaper", "I'm starting from Polonnaruwa"), - save trips, track expenses and see budget status, - browse an explore section of districts, listings and events.

Admins verify listings and events and manage the AI model configuration.

**Real data in place:** 25 districts and 6,572 listings, from OpenStreetMap, Booking.com, Ticketmaster and Wikidata, plus admin-reviewed entries (events and entry fees from a scraper).

## 2. Architecture



| Layer              | Tech                                                          | Role                                                                                                                                         |
|--------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| frontend-web       | Next.js, Zustand, next-auth                                   | UI: chat panel, trip view, budget, explore, admin. Talks <b>only</b> to NestJS.                                                              |
| backend            | NestJS, Prisma, raw-SQL migrations                            | Auth guards, users, trips, budget, chat, explore, admin. Proxies planning requests to the AI backend. It is the <b>only trust boundary</b> . |
| ai-backend         | FastAPI, LangGraph                                            | Planner and recommendation agents, scoring, clustering, budget, deterministic fallback planner, output validation, follow-up edits.          |
| Keycloak           | Realm with <code>admin</code> and <code>traveler</code> roles | Login, JWT issuing, email verification (Mailpit locally).                                                                                    |
| Postgres + PostGIS | Docker                                                        | Users, itineraries, expenses, listings, events. Schema owned by SQL files run by <code>backend/db/migrate.py</code> .                        |
| frontend-mobile    | Flutter                                                       | <b>Not started.</b> Out of scope.                                                                                                            |

### Key design ideas (good to mention)

- **LLM failover chain.** Gemini is primary and Groq is the failover. If both fail, a **deterministic fallback planner** returns a valid, budget-checked plan, so the user never gets a server error.
- **Output is validated, not trusted.** The planner's output must be schema-valid and within budget. Tests assert properties, not exact text.
- **Data isolation.** Every query is scoped by `user_id`. A foreign resource returns 404, so it looks the same as one that doesn't exist.
- **Admin-managed AI config.** The model chain and API keys are managed in Admin > AI models (stored in the DB, with `.env` as the fallback).

### Status per repo (from `REPO_STATUS.md` ; later work from project notes)

| Repo            | State                                                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ai-backend      | Feature-complete through its master-plan phases. The fallback planner is complete. The ReAct LLM path is built, but free-tier limits mean it often falls back. |
| backend         | Auth, users, chat, trips, explore, budget built. Admin and scraper modules built after the 09-23 survey (see Master Test Plan: 8 modules, about 50 routes).    |
| frontend-web    | All main screens built against the real API, plus the 2026-10-01 motion/landing redesign. Working branch is <code>new-main</code> .                            |
| frontend-mobile | Not started.                                                                                                                                                   |

**Honest limitations:** Gemini and Groq free-tier quotas cap how often the full LLM path runs. Restaurants have no free image source (a placeholder is shown). Ticketmaster has no Sri Lanka events, so events come from admin entry and the scraper.

---

## 3. Workflows

### 3.1 User workflow (what the demo follows)

1. **Register / sign in** through Keycloak and verify email (Mailpit locally).
2. **Describe a trip** in the chat. The frontend calls NestJS, which calls the AI backend.
3. **AI orchestration**: resolve places, check weather, disaster alerts and free days, score and cluster listings, build days, apply the budget, validate, and fall back if needed.
4. **View the itinerary** by day (map, costs, status against budget).
5. **Refine by follow-up** ("make day 2 cheaper"). Only the targeted day is rebuilt, and the other days stay byte-identical.
6. **Save the trip, add expenses** and watch budget status (computed against three thresholds).
7. **Explore** districts, listings and events.
8. **Admin** (separate login): verify or reject listings and events, view stats, and configure AI models.

### 3.2 Developer workflow (CI/CD)

Each repo has its own GitHub Actions workflow at `.github/workflows/ci.yml`.

| Repo                | Triggers                                                          | Test job                                                                                                                                | Deploy job                                                                                                                                                                                                                                        |
|---------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>backend</b>      | push to <code>main</code> , PRs, manual                           | Node 26: <code>npm ci</code> → <code>prisma generate</code> → <code>lint</code> → <code>npm test</code> → <code>npm run test:e2e</code> | On <code>main</code> only, after tests pass: builds images to GHCR tagged with the commit SHA (db/keycloak images only when their files change), then deploys. Per the notes the target is the Lightsail server ( <b>verify</b> the deploy step). |
| <b>ai-backend</b>   | push to <code>main</code> , PRs, manual                           | Python 3.12: <code>pip install</code> → <code>python -m pytest -m "not external" -q</code><br>(placeholder API keys, no network)        | On <code>main</code> only: builds the image to GHCR, then deploys.                                                                                                                                                                                |
| <b>frontend-web</b> | push to <code>main</code> and <code>new-main</code> , PRs, manual | Node 22: <code>npm ci</code> → <code>lint</code> → <code>npm test</code>                                                                | On <code>new-main</code> only, after tests pass: deploys to <b>Vercel</b> via CLI. It skips cleanly if the Vercel secrets are not set.                                                                                                            |

Principles to state: - **Tests gate deployment**. A commit that fails lint or tests never reaches production. - CI uses no real secrets and no live services. - Playwright and the 12 golden scenarios are run manually against the full stack before a release.

### 3.3 Local run order (`RUNNING.md`)

1. `docker compose up -d` in `backend/` (Postgres/PostGIS, Redis, Keycloak, Mailpit). Wait for Keycloak.
2. Apply migrations: `python db\migrate.py --status`, then `python db\migrate.py`.
3. NestJS: `npm install`, `npx prisma generate`, `npm start` (port 3001).
4. AI backend: activate the venv, then `python -m uvicorn main:app --port 8000`.

5. Frontend: `npm install`, `npm run dev` (port 3000).

---

## 4. How to present the demo

---

### 4.1 Preparation (the day before and an hour before)

- [ ] Bring the whole stack up in the order above. **Do a full dry run, start to finish.**
- [ ] Run `python db\migrate.py --status` so no migration is pending. A skipped migration fails silently at runtime.
- [ ] Check `GEMINI_API_KEY` quota. Free-tier limits are the biggest live-demo risk.
- [ ] Pre-register two accounts and verify their email: a **traveler** and an **admin** (Keycloak accounts are per-machine, and the realm export has no users).
- [ ] Remember the 12-character password cap, and that verification links expire after 5 minutes.
- [ ] Verify listings through the admin flow, or the planner returns little or nothing (it filters on verified listings).
- [ ] Do one planning run beforehand so the data is warm, and keep a **screen recording** of a good run as backup.
- [ ] Open tabs ahead of time: app (3000), Mailpit (8025), Keycloak (8081), test reports, GitHub Actions.
- [ ] Google sign-in is **not configured**. Use email/password and don't click it.

### 4.2 Suggested running order (about 10–12 minutes)

| Min   | Segment                       | What to show and say                                                                                                                                                                             |
|-------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0–1   | <b>Problem and vision</b>     | Planning a Sri Lanka trip means juggling budget, weather and distances. SmartJourney plans it from a chat.                                                                                       |
| 1–2   | <b>Architecture slide</b>     | The diagram in §2. Stress that the frontend talks only to NestJS and that the AI backend is isolated.                                                                                            |
| 2–3   | <b>Landing page and login</b> | Show the redesigned landing page. Sign in through Keycloak.                                                                                                                                      |
| 3–6   | <b>Core demo: plan a trip</b> | Type a realistic request with a budget. Narrate while it loads (5–20 s): the multi-agent pipeline, weather, disasters and budget checks. Show the day-by-day result and the cost against budget. |
| 6–7   | <b>Follow-up edit</b>         | "Make day 2 cheaper." Show that only day 2 changed and the other days are identical.                                                                                                             |
| 7–8   | <b>Save and budget</b>        | Save the trip, add an expense and show the budget status change.                                                                                                                                 |
| 8–9   | <b>Explore</b>                | Districts, listings and images.                                                                                                                                                                  |
| 9–10  | <b>Admin</b>                  | Sign in as admin: verify a listing, show Admin > AI models and the stats.                                                                                                                        |
| 10–12 | <b>Quality evidence</b>       | Open the tests brief: 637 automated tests, CI green, golden scenarios 10/12, and the security isolation tests. State the known gaps.                                                             |
| 12+   | <b>Roadmap and Q&amp;A</b>    | Mobile app, closing the known gaps, a paid LLM tier for a better planner.                                                                                                                        |

### 4.3 Speaking tips

- Lead with the **user value**, then explain the technology.
- Narrate during the LLM wait, because silence feels like a crash.
- If the AI is slow or falls back, **say so deliberately**: "this is the deterministic fallback working as designed, so the user never gets an error." It's a strength, not an embarrassment.
- Quote measured numbers only: 10/12, not 11/12.
- Be upfront about what isn't done (mobile app, billing, authenticated Playwright journeys). Evaluators trust honest scope.

### 4.4 If something breaks live

| Problem                                   | Response                                                                                                                                                       |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gemini or Groq rate-limited or timing out | The fallback plan appears. Explain the failover chain. If the plan comes back empty, switch to the recording.                                                  |
| Keycloak not ready or login fails         | Check <code>http://localhost:8081/realms/smartjourney/.well-known/openid-configuration</code> . Use the pre-verified account.                                  |
| Empty results                             | Listings aren't verified. Run the admin verify flow.                                                                                                           |
| Follow-up edit crashes                    | This is the known orchestrator defect (golden scenario 6). Skip the follow-up, and say it was found by testing. <b>Verify</b> whether it has been fixed since. |
| Total failure                             | Play the screen recording and walk through the test evidence.                                                                                                  |

### 4.5 Likely questions and honest answers

- **"How do you know the AI output is correct?"** It is non-deterministic, so we validate the schema and budget, check the plan source, and run 12 golden scenarios. Result: 10/12.
- **"What if the LLM is down?"** A deterministic fallback plan returns HTTP 200. This is tested.
- **"Is user data safe?"** Every query is scoped by `user_id`, foreign access gets 404, and JWT and role guards are covered by e2e tests. The AI backend has no auth of its own, so it relies on network isolation (a documented residual risk).
- **"Why is coverage only 70–75%?"** Coverage is reported, not gated, on a young suite. We prioritised ownership and failure paths.
- **"What's next?"** The mobile app, extending performance and load baselines, scripted Keycloak login for the Playwright journeys, and fixing the open orchestrator defect.

---

## 5. Pre-evaluation checklist

- [ ] Re-run `python -m pytest -m "not external" -q` and `python scripts/e2e_check.py`, and update the test numbers (the test file counts on disk differ from the plan).
- [ ] Confirm whether the orchestrator `_verify_node` crash is fixed.
- [ ] Confirm the deployment status for **verify** items (is the live site up, and which URL).
- [ ] Do the full dry run with a timer.

- [ ] Keep `PROGRESS_EVALUATION_TESTING_BRIEF.md` open for the testing questions.